

## **M2M - Modern Management (Ristorante)**

### **Sistema full-stack realtime per la gestione digitale di ristoranti moderni**

#### **📄 Descrizione progetto**

Il progetto permette di creare un sistema di comunicazione tra servizi indipendenti, simulando un'architettura a microservizi semplificata.

Ogni modulo può inviare e ricevere dati in tempo reale attraverso un sistema centralizzato, permettendo lo scambio di informazioni tra componenti diversi del sistema.

#### **📄 Obiettivo del progetto**

L'obiettivo del progetto è realizzare una piattaforma modulare e scalabile che permetta la comunicazione tra più servizi in tempo reale, utilizzando tecnologie moderne come Node.js e Supabase.

Il progetto mira a simulare un'infrastruttura backend reale, utile per comprendere il funzionamento dei sistemi distribuiti.

#### **📄 Problema che risolve**

Molti sistemi moderni hanno difficoltà a far comunicare tra loro diversi servizi in modo semplice e veloce.

Questo progetto risolve il problema creando un sistema centralizzato che permette lo scambio di dati in tempo reale tra moduli diversi, riducendo la complessità dell'integrazione tra servizi.

#### **📄 Overview del progetto**

M2M è una piattaforma web progettata per digitalizzare e ottimizzare la gestione operativa di un ristorante, sostituendo processi manuali (carta, comande tradizionali) con un sistema centralizzato, realtime e multi-dispositivo.

Il sistema permette la gestione simultanea di tavoli, ordini e sala, sincronizzando ogni modifica in tempo reale tra camerieri e amministratori.

L'obiettivo non è solo la gestione, ma la simulazione di un ambiente reale di ristorazione con logiche di comunicazione event-driven.

## 📋 Architettura del sistema

Il progetto è basato su un'architettura full-stack moderna event-driven:

- Frontend Next.js (App Router): interfaccia operativa per staff e admin
- Backend Supabase: database, autenticazione e realtime
- State Layer (Zustand): gestione locale del carrello e tavoli
- Server Actions: operazioni sicure lato server
- Realtime Engine: sincronizzazione immediata tra dispositivi

Ogni azione (ordine, apertura tavolo, modifica menu) viene propagata in tempo reale a tutti i client connessi.

## 🌟 Funzionalità

### Area Camerieri

- Mappa interattiva della sala con stato tavoli
- Apertura e gestione tavolo in tempo reale
- Creazione ordini con sistema a carrello
- Aggiunta note personalizzate per ogni piatto
- Aggiornamenti live senza refresh pagina

### Area Amministratore

- Gestione completa del menu (categorie e piatti)
- Gestione utenti con ruoli (admin / staff)
- Monitoraggio ordini attivi e completati
- Dashboard con panoramica operativa
- Controllo stato tavoli in tempo reale

### Funzionalità tecniche

- Autenticazione sicura con Supabase Auth
- Aggiornamenti realtime multi-dispositivo
- UI responsive ottimizzata per tablet e mobile
- Validazione dati con Zod
- Architettura modulare e scalabile

## 📋 Tecnologie utilizzate

- Next.js 16: frontend e backend (App Router)
- TypeScript: tipizzazione e robustezza
- Supabase: database, auth e realtime
- Tailwind CSS: UI moderna e responsive
- Zustand: gestione stato locale
- Zod: validazione dati

- Lucide React: iconografia UI

## 🔗 Modello dati (semplificato)

- profiles → utenti e ruoli sistema
- tables → tavoli del ristorante
- orders → ordini attivi e storici
- order\_items → singoli elementi ordine
- menu\_categories → categorie menu
- menu\_items → piatti e prezzi

## 🔗 UI e UX

Il sistema è progettato con approccio mobile-first, ottimizzato per:

- Tablet per camerieri
- Desktop per amministrazione
- Interazioni rapide senza reload
- Feedback visivo immediato

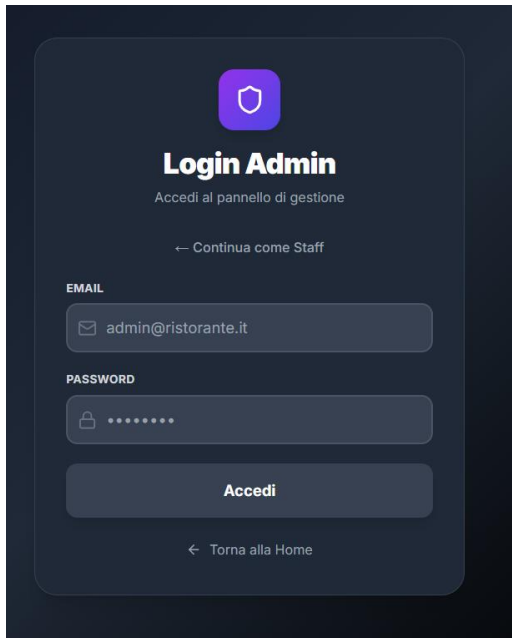
## 🔗 Screenshot dell'applicazione

Di seguito sono riportate alcune immagini del sistema M2M che mostrano le principali funzionalità e interfacce utente.

## 🔗 Home Page



## ? Login Admin



The Login Admin interface features a dark blue background. At the top center is a purple shield icon. Below it, the text "Login Admin" is displayed in white, followed by "Accedi al pannello di gestione" in a smaller font. A link "← Continua come Staff" is positioned above the email input field. The email field is labeled "EMAIL" and contains the text "admin@ristorante.it". The password field is labeled "PASSWORD" and contains seven dots. A dark blue "Accedi" button is centered below the password field. At the bottom, a link "← Torna alla Home" is displayed.

**Login Admin**  
Accedi al pannello di gestione

← Continua come Staff

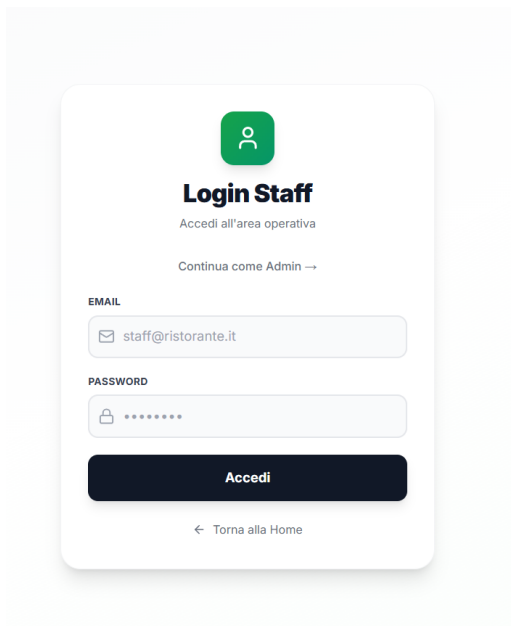
EMAIL  
admin@ristorante.it

PASSWORD  
.....

**Accedi**

← Torna alla Home

## ? ? Login Staff



The Login Staff interface has a light gray background. It features a green person icon at the top center. Below the icon, the text "Login Staff" is shown in bold, followed by "Accedi all'area operativa" in a smaller font. A link "Continua come Admin →" is located above the email input field. The email field is labeled "EMAIL" and contains the text "staff@ristorante.it". The password field is labeled "PASSWORD" and contains seven dots. A dark blue "Accedi" button is centered below the password field. At the bottom, a link "← Torna alla Home" is displayed.

**Login Staff**  
Accedi all'area operativa

Continua come Admin →

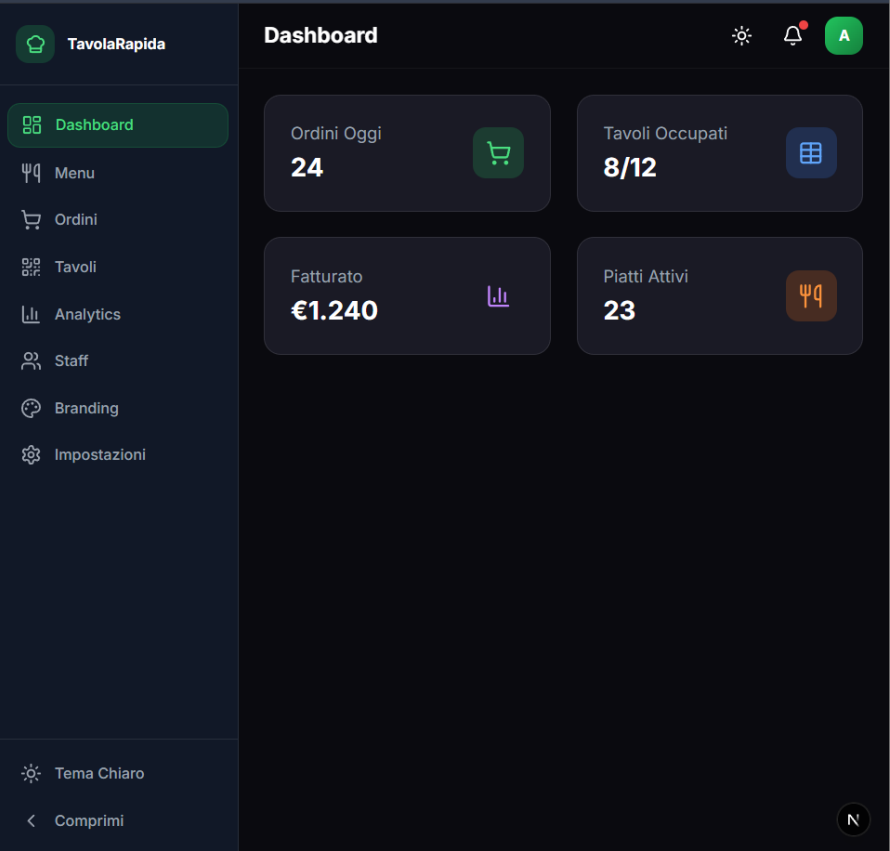
EMAIL  
staff@ristorante.it

PASSWORD  
.....

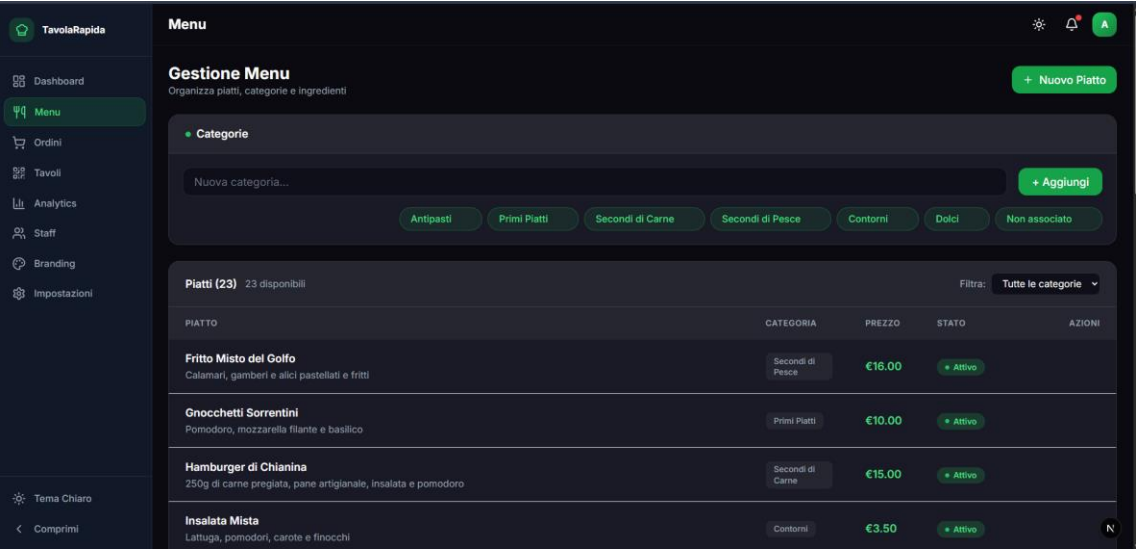
**Accedi**

← Torna alla Home

## ? Dashboard



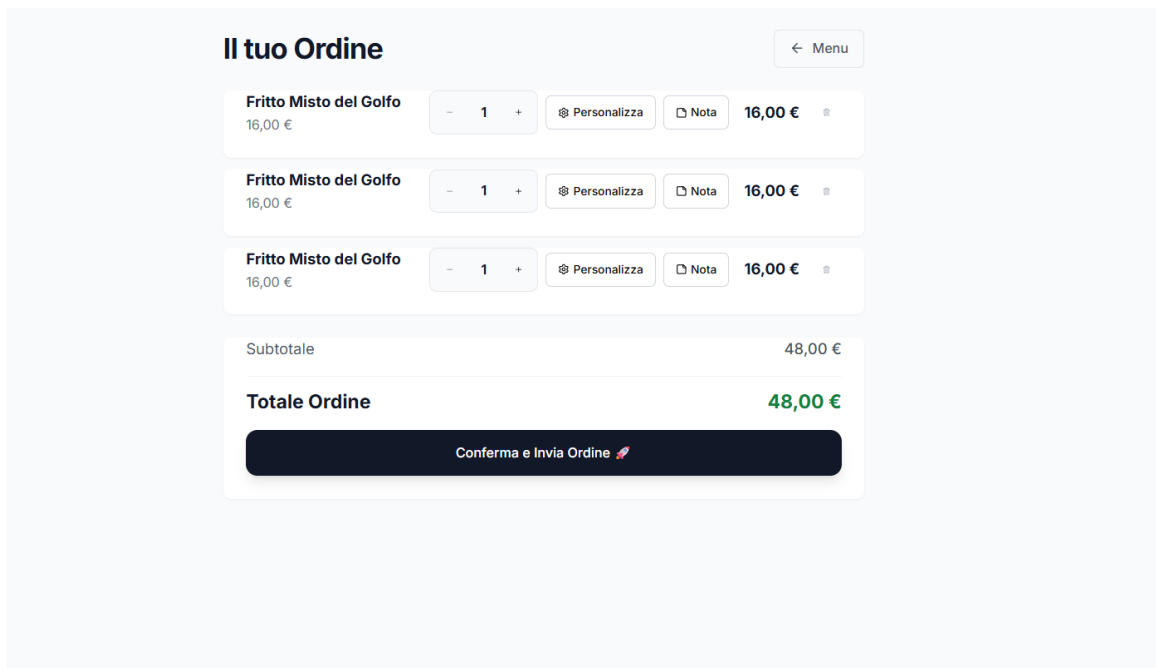
## Menu



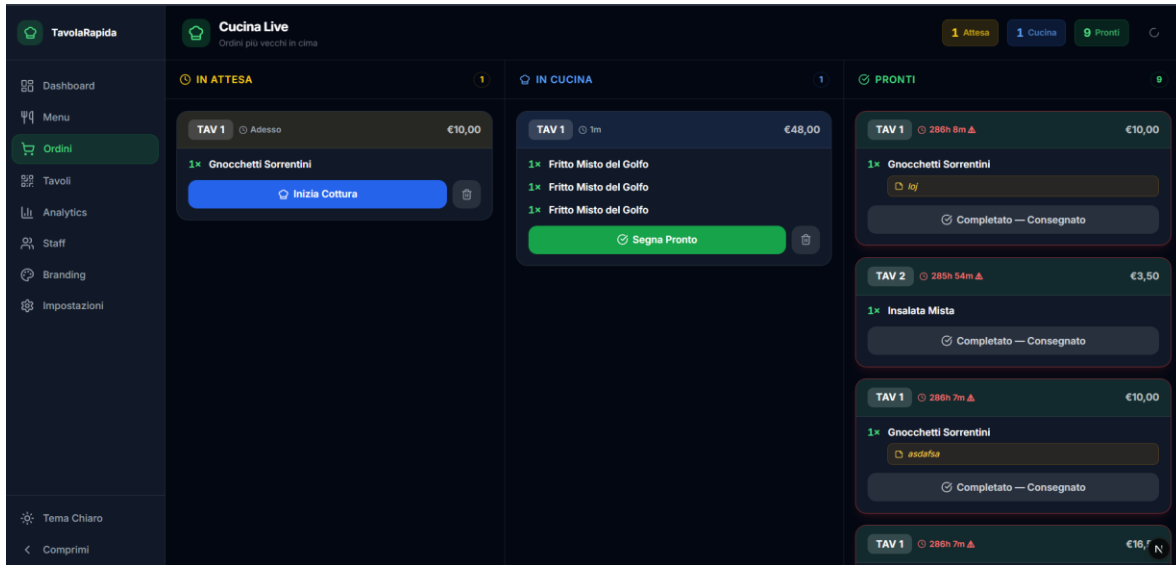
## Ordinazione



## [Carrello Ordini](#)



## [Kitchen View](#)



## 📖 Installazione

1. Clonare il repository:  
git clone <https://github.com/montronechris/m2m.git>
2. Entrare nella cartella:  
cd m2m
3. Installare le dipendenze:  
npm install
4. Configurare ambiente:  
cp .env.example .env.local
5. Avviare il progetto:  
npm run dev

## 📖 Requisiti

- Node.js 20 o superiore
- Account Supabase attivo

## 📖 Configurazione

Nel file .env.local inserire:

- URL del progetto Supabase
- API Key pubblica (anon key)
- API Key privata (service role, se richiesta)
- Configurazione autenticazione (Auth)
- Abilitazione realtime database

Queste variabili permettono la connessione tra frontend e backend Supabase.

### 🔗 Scelte progettuali

- Separazione tra UI, logica e servizi
- Architettura modulare e scalabile
- Aggiornamenti realtime senza refresh
- Stato ibrido (Zustand + Supabase)
- Sicurezza tramite Server Actions

L'obiettivo è simulare un sistema reale utilizzabile in ambito professionale.

### 🔗 Competenze dimostrate

- Sviluppo full-stack end-to-end
- Uso di database realtime
- Architetture event-driven
- UI/UX moderne e responsive
- Gestione autenticazione e ruoli
- Scrittura di codice scalabile e mantenibile

### 🔗 Possibili evoluzioni

- Pagamenti digitali integrati
- Stampanti fiscali e termiche
- Dashboard con KPI e analytics
- Modalità offline (PWA)
- AI per suggerimenti menu
- Sistema gestione magazzino

### 🔗 Nota finale

Progetto sviluppato come capolavoro finale del percorso scolastico, con l'obiettivo di dimostrare competenze nello sviluppo di applicazioni reali, moderne e scalabili.

**Autore:** Montrone Christian

**Anno:** 2026